

Examen Laravel

EX-ZCCW-WAJR

Tema: Tienda de Productos | Fecha: 22 de abril de 2026 | Duración: 120 minutos |
Puntaje total: 100 pts | Cédula: 1056123379 | Recargas: 16 | ID: EX-ZCCW-WAJR

01 **Gestión de Product — Tienda de Productos**

25 pts

El sistema **Tienda de Productos** requiere gestionar el recurso **Product**.
Se espera que la aplicación permita registrar nuevos elementos, consultarlos, modificarlos y retirarlos. Los registros retirados **no deben eliminarse de la base de datos**: deben permanecer almacenados y poder recuperarse si fuera necesario.

Campos requeridos

La tabla `products` debe contemplar los siguientes campos:

- `name` — tipo *string*
- `price` — tipo *decimal*
- `category_id` — tipo *integer*
- `is_active` — tipo *boolean*

Funcionalidades esperadas

- Registrar un nuevo **Product** a través de un formulario.
- Mostrar el listado de elementos disponibles, incluyendo únicamente los registros activos.
- Consultar el detalle de un elemento individual.
- Editar la información de un elemento existente.
- Retirar un elemento del sistema sin eliminarlo físicamente de la base de datos.

RÚBRICA DE CALIFICACIÓN

0

No hay CRUD funcionando.

1

Solo funciona una parte del CRUD y tiene varios errores.

2

CRUD básico: crea y consulta, pero falta completar bien editar/eliminar lógico.

3

Hay avance en las cuatro operaciones, pero varias fallan o quedan a medias.

4

CRUD casi completo: falta pulir una operación o hay detalles menores en listados/detalles.

5

CRUD completo: las cuatro operaciones funcionan bien de punta a punta.

02 Relaciones

15 pts

El sistema requiere que el recurso **Category** esté relacionado con **Product**.

Cada producto pertenece a una categoría. Una categoría puede tener muchos productos.

Relación entre recursos

- La tabla `products` debe tener la clave foránea `category_id` .

Funcionalidad esperada

Debe ser posible acceder a los datos relacionados a través de la siguiente ruta:

GET `/categories/{id}/products`

RÚBRICA DE CALIFICACIÓN

0

No implementa la relación entre modelos.

1

La relación está mal definida y no permite consultar bien.

2

La relación existe, pero la consulta es parcial o falla en algunos casos.

3

Hay modelos y relación encaminados, pero la consulta o la ruta fallan en parte.

4

Relación casi completa: la consulta principal funciona; faltan detalles o un caso borde.

5

Relación completa: modelos, consulta y respuesta acordes al enunciado.

03

Middleware — Control de acceso por propiedad: solo el dueño puede modificar

25
pts

Control de acceso por propiedad: solo el dueño puede modificar

Un recurso (publicación, producto, etc.) solo puede ser modificado o eliminado por el usuario que lo creó.

El Middleware debe comparar el usuario autenticado con el propietario del recurso.

Ruta que debe quedar protegida

PUT /posts/5

Comportamiento esperado del Middleware

- El usuario autenticado envía una petición PUT o DELETE a un recurso.
- El Middleware extrae el ID del recurso de la URL.
- Se consulta el recurso en la base de datos para obtener su `user_id` .
- Se compara `user_id` del recurso con `Auth::id()` .
- Si no coinciden, se retorna HTTP 403.
- Si coinciden, el usuario es el propietario y la petición continúa.

RÚBRICA DE CALIFICACIÓN

0

No implementa el middleware.

1

Middleware creado, pero no valida correctamente.

2

Valida algunas condiciones, pero no controla bien el acceso.

3

Valida parte de lo pedido, pero el bloqueo o las rutas no quedan bien cerrados.

4

Middleware casi completo: cumple lo esencial; falta pulir un caso o el mensaje de respuesta.

5

Middleware completo: condiciones, rutas y permitir/bloquear según el enunciado.

04 API Externa — Máximo, Mínimo y Rango

25 pts

Se debe consumir el siguiente endpoint externo con **JavaScript (fetch GET)** desde una **vista Blade** y procesar los datos obtenidos según se describe.

Endpoint a consumir

GET <https://jsonplaceholder.typicode.com/albums>

Tarea a realizar

Consumir el endpoint con `fetch` y calcular el **valor máximo y mínimo del campo id**. Retornar ambos valores junto con la diferencia entre ellos.

JSON que retorna el endpoint (array)

```
[
  {
    "id_maximo": 100,
    "id_minimo": 1,
    "diferencia": 99
  }
]
```

Requisitos

- La petición debe realizarse con `fetch` usando método `GET` en una vista Blade.
- Se debe mostrar el resultado del procesamiento en la misma vista.
- Se debe contemplar el caso en que la API externa no responda correctamente.
- El resultado debe retornarse en formato JSON.

RÚBRICA DE CALIFICACIÓN

0

No consume la API ni muestra resultados.

1

Intenta consumir la API, pero falla la petición o la vista.

2

Consume la API, pero el procesamiento de datos queda incompleto.

3

Hay `fetch` en Blade y datos, pero el resultado no cumple bien lo pedido.

4

Casi completo: procesa y muestra bien; falta un detalle del filtrado, orden o formato.

5

Completo: `fetch GET` en Blade, procesamiento y salida según el enunciado.

05 Git — Historial de commits con estructura semántica

10 pts

Historial de commits con estructura semántica

El repositorio en GitHub debe contener **al menos 8 commits**, con mensajes descriptivos que reflejen el progreso real del desarrollo.

Requisitos a cumplir

- Mínimo 8 commits en la rama principal.
- Los mensajes deben seguir la convención: `tipo: descripción` .
- Tipos válidos: feat, fix, refactor, docs, test, chore.
- Cada commit debe representar un avance atómico y significativo.

Ejemplo esperado

```
feat: inicializar proyecto Laravel con configuración base
feat: crear migración y modelo para posts
feat: implementar CRUD completo de posts
feat: agregar relación User hasMany Post
feat: crear middleware de autenticación por rol
feat: consumir API externa con fetch en Laravel
refactor: optimizar consultas en PostController
docs: actualizar README con instrucciones de instalación
```

Cómo se evaluará

- Se revisará el repositorio privado en GitHub.
- Se verificará el historial de commits, ramas y Pull Requests.
- El enlace al repositorio debe entregarse junto con el proyecto.

RÚBRICA DE CALIFICACIÓN

0

No hay evidencia de uso de Git/GitHub.

1

Hay pocos commits y desorden en el historial.

2

Commits básicos, pero con manejo limitado de ramas.

3

Hay commits y ramas, pero el flujo o el PR no se entiende del todo.

4

Casi completo: rama(s), commits y PR con buen orden; falta algo para revisarlo sin dudas.

5

Completo: historial claro, ramas y PR alineados con lo pedido en el enunciado.

25

CRUD

15

RELACIONES

25

MIDDLEWARE

25

API EXTERNA

10

GIT

100

TOTAL